# Training a Control Barrier Function in Latent Space

## Training the CBF Function $B_\theta(z, o)$

### 1. Objective

Learn a **neural control barrier function (CBF)** in latent space:

$$B_\theta(z, o)$$

- Input: latent state $z$, obstacle representation $o$
- Output: scalar barrier value
- Convention:
  - $B_\theta(z, o) \geq 0 \rightarrow$ **Safe**
  - $B_\theta(z, o) < 0 \rightarrow$ **Unsafe**

---

### 2. Data Requirements

#### (A) State Safety Labels

- Samples: $(z, o)$
- Labels:
  - Safe (collision-free)
  - Unsafe (collision)
- Source:
  - Existing dataset (`robot_obs_dataset.py`)
  - Geometric collision checker (MoveIt / Robo3D)

---

#### (B) Transition Pairs (Critical for CBF)

- Samples:

$$(z_k, z_{k+1}^{nom}, o)$$

- Where:
  - $z_{k+1}^{nom}$ is obtained from **planner update using only Goal + Prior losses**
- Purpose:
  - Enforce **discrete-time CBF constraint**

---

**(C) Boundary Coverage (Important)**

- Include **hard samples near obstacle boundaries**
- Reason:
  - Prevent weak/over-smooth barrier near decision boundary
  - Improve safety robustness

---

## 3. Generating Transition Data

Transitions are **not directly available** → must be generated.

**Procedure**

1. Run existing latent planner
2. Modify planner:
   - Use **Goal + Prior loss only**
   - Remove collision loss
3. During rollout, record:

$$(z_k, z_{k+1}^{nom}, o)$$

**Key Insight**

- This matches **inference-time behavior**:
  - Planner → produces $z^{nom}$
  - CBF → enforces safety

---

## 4. Model Architecture

- Use a **MLP**
- Input:

$$[z, o]$$

- Output:

$$B_\theta(z, o)$$

Optional design choices:

- Normalize latent input
- Encode obstacles compactly (e.g., center + radius)

---

## 5. Training Objective

### Full Loss Function

$$\mathcal{L} = \lambda_s \mathbb{E}_{\mathcal{S}}[\max(-B_\theta(z,o),0)] + \lambda_u \mathbb{E}_{\mathcal{U}}[\max(B_\theta(z,o),0)] + \lambda_d \mathbb{E}[\max((1-\alpha\Delta)B_\theta(z_k,o) - B_\theta(z_{k+1}^{nom},o),0)]$$

---

### Interpretation of Terms

### (1) Safe Region Constraint

$$\max(-B_\theta, 0)$$

Penalizes safe samples classified as unsafe.

### (2) Unsafe Region Constraint

$$\max(B_\theta, 0)$$

Penalizes unsafe samples classified as safe.

### (3) Discrete CBF Constraint (Core Term)

$$B(z_{k+1}^{nom}, o) \geq (1 - \alpha\Delta)\, B(z_k, o)$$

- Ensures **forward invariance of safety**
- Enforces:
  - If safe → stay safe after update
  - Prevents unsafe drift

---

### Optional Enhancements

- Binary cross-entropy (BCE) for classification calibration
- Margin $\gamma$:
  - Safe: $B \geq \gamma$
  - Unsafe: $B \leq -\gamma$

---

## 6. Training Algorithm (Pipeline)

### Step-by-step

1. **Freeze VAE encoder**
2. **Prepare dataset**
   - Encode states: $x \to z$
   - Attach obstacle info $o$
   - Add safety labels
3. **Generate transitions**

- Run planner (Goal + Prior only)
- Record $(z_k, z_{k+1}^{nom}, o)$

4. **Initialize $B_\theta$ (MLP)**
5. **Training loop**
   - Sample minibatch
   - Compute:
     - Safe loss
     - Unsafe loss
     - CBF constraint loss
   - Backpropagation (Adam)
6. **Validation**
   - Run rollouts
   - Check collisions via geometric checker
7. **Hyperparameter tuning**
   - $\lambda_s, \lambda_u, \lambda_d$
   - $\alpha, \Delta$

---

## 7. Runtime Integration (CBF2 Step)

At inference:

1. Compute nominal update:
$$z_{k+1}^{nom}$$
(from Goal + Prior planner)
2. Evaluate barrier gradient:
$$d = \nabla_z B_\theta(z_{k+1}^{nom}, o)$$
3. Compute correction (CBF2)
4. Apply correction:
$$z_{k+1}^{safe}$$
5. Continue:
$$z \leftarrow z_{k+1}^{safe}$$

---

## 8. Key Design Principle

### Separation of Responsibilities

| Component | Role |
|---|---|
| Planner | Goal reaching + latent prior |
| CBF | Safety enforcement |

---

## 9. Critical Insights (Research-Level)

- Transition consistency is **essential** → mismatch breaks guarantees
- Removing collision loss from planner is **intentional**, not a limitation
- CBF learns **geometry of safe set**, not just classification
- Quality of boundary samples determines **real-world safety performance**

---

### Final Takeaway

You are effectively transforming your system from:

**"Optimization-based planner with soft safety penalties"**

→

**"Two-layer system: nominal planner + formally constrained safety filter (CBF)"**